APCS 程式設計系列 · U07 · 實作中級 · C++ / Python 對照版

函式與遞迴 重點講義

函式與回傳值 · 傳值與傳參考 · 區域與全域變數 · 遞迴三要素 · 遞迴追蹤 · 呼叫次數爆炸 · 河內塔與分治

1. 函式：把一件事包起來，給它名字

C++ 三個數取最大：自己寫函式

```cpp
#include <bits/stdc++.h>
using namespace std;

int max2(int a, int b) {      // 回傳型別 函式名稱(參數)
    if (a > b) return a;
    return b;                  // return：把答案交回去
}

int max3(int a, int b, int c) {
    return max2(max2(a, b), c);   // 函式可以呼叫函式
}

int main() {
    int x, y, z;
    cin >> x >> y >> z;
    cout << max3(x, y, z) << "\n";
}
```

Python

```python
def max2(a, b):           # def 函式名稱(參數):
    if a > b:
        return a
    return b              # return：把答案交回去

def max3(a, b, c):
    return max2(max2(a, b), c)   # 函式可以呼叫函式

x, y, z = map(int, input().split())
print(max3(x, y, z))
```

輸入	輸出
4 9 7	9

- **參數**：呼叫時把值「交進去」；**回傳值**：函式算完把答案「交回來」。
- 執行到 return 就立刻離開函式，後面的程式不會執行。
- 沒有回傳值：C++ 寫 void；Python 沒寫 return 會回傳 None。兩種語言都要先定義函式再呼叫。

陷阱提醒

【錯】忘了 return：C++ 回傳不確定的值，Python 回傳 None。

【錯】把 print 當成 return：函式裡印出答案，不代表呼叫的地方拿得到答案。

2. 傳值與傳參考：函式裡改了，外面會不會變？

C++ 改得到外面的，和改不到的

```cpp
#include <bits/stdc++.h>
using namespace std;

void byValue(int x) { x = 100; }      // 改的是複本
void byRef(int &x) { x = 100; }       // & 參考：改的就是本人
void setArr(int a[]) { a[0] = 100; }  // 陣列：改得到外面

int main() {
    int p = 1, q = 1, arr[3] = {1, 2, 3};
    byValue(p);
    byRef(q);
    setArr(arr);
    cout << p << " " << q << " " << arr[0] << "\n";
}
```

Python

```python
def change_int(x):
    x = 100          # 只改到函式裡的 x
def change_list(a):
    a[0] = 100       # 改的是同一個 list，外面看得到
def replace_list(a):
    a = [100]        # a 改指向新的 list，外面不受影響

p, arr1, arr2 = 1, [1, 2, 3], [1, 2, 3]
change_int(p)
change_list(arr1)
replace_list(arr2)
print(p, arr1[0], arr2[0])
```

輸入	輸出
（不需要輸入）	C++： 1 100 100 Python： 1 100 1

傳進去的東西	C / C++	Python
整數、小數	傳值：改不到外面	改不到外面（重新指定只改到函式裡）
陣列 / list	改 a[i] 改得到外面	a[i] = ...、append 改得到外面
想改外面的整數	參數寫成 int &x（C++）	用 return 把新值傳回來
重新指定整個	（陣列不能整個指定）	a = [...] 外面不受影響

陷阱提醒

【錯】C++ 的 swap 沒用 & 就換不到；Python 的 a = a + [3] 是新的 list，外面沒變。

3. 區域變數與全域變數

C++ 全域計數器：記錄函式被呼叫幾次

```cpp
#include <bits/stdc++.h>
using namespace std;

int cnt = 0;            // 全域變數：所有函式共用

void hit() {
    cnt++;              // 改到的是全域的 cnt
    int t = 5;          // 區域變數：函式結束就消失
}

int main() {
    for (int i = 0; i < 3; i++) hit();
    cout << cnt << "\n";
}
```

Python

```python
cnt = 0                 # 全域變數

def hit():
    global cnt          # 要修改全域變數，先宣告 global
    cnt += 1
    t = 5               # 區域變數：函式結束就消失

for i in range(3):
    hit()
print(cnt)
```

輸入	輸出
（不需要輸入）	3

- **區域變數**：在函式裡建立，函式結束就消失；每次呼叫都是全新的一份（遞迴的每一層也各有一份）。
- **全域變數**：寫在所有函式外面，大家共用，常拿來計數或存大陣列。
- 同名時用的是區域變數。Python 要修改全域變數必須先寫 global；C++ 直接改就是全域的。

陷阱提醒

【錯】函式裡寫 cnt += 1 卻沒寫 global cnt：會出現 UnboundLocalError。

4. 遞迴：函式呼叫自己

遞迴三要素

- **終止條件**：最小的情況直接回答，不再呼叫自己（沒寫就停不下來）。
- **縮小問題**：呼叫自己，但參數要更接近終止條件（n 變成 n - 1、n / 2……）。
- **組合答案**：用小問題的答案，算出大問題的答案。

C++ 階乘：n! = n × (n-1)!

```cpp
#include <bits/stdc++.h>
using namespace std;

long long fact(int n) {
    if (n == 0) return 1;        // 1. 終止條件
    return n * fact(n - 1);      // 2. 縮小問題 3. 組合答案
}

int main() {
    cout << fact(5) << " " << fact(20) << "\n";
}
```

Python

```python
def fact(n):
    if n == 0:
        return 1                 # 1. 終止條件
    return n * fact(n - 1)       # 2. 縮小問題 3. 組合答案

print(fact(5), fact(20))
```

輸入	輸出
（不需要輸入）	120 2432902008176640000

C++ 輾轉相除法：gcd(a, b) = gcd(b, a % b)

```cpp
#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a, long long b) {
    if (b == 0) return a;        // 餘數是 0，a 就是答案
    return gcd(b, a % b);
}

int main() {
    long long a, b;
    cin >> a >> b;
    long long g = gcd(a, b);
    cout << g << " " << a / g * b << "\n";   // 最小公倍數：先除後乘
}
```

Python

```python
def gcd(a, b):
    if b == 0:
        return a                 # 餘數是 0，a 就是答案
    return gcd(b, a % b)

a, b = map(int, input().split())
g = gcd(a, b)
print(g, a // g * b)             # 最小公倍數：先除後乘
```

輸入	輸出
84 36	12 252

追蹤：gcd(84, 36) → gcd(36, 12) → gcd(12, 0) → 回傳 12。每一步數字至少減半，所以很快。

5. 遞迴追蹤：程式識讀最核心

C++ print 在遞迴前、在遞迴後

```cpp
#include <bits/stdc++.h>
using namespace std;

void f(int n) {
    if (n == 0) return;
    cout << "in" << n << " ";     // 呼叫前：往下走時印
    f(n - 1);
    cout << "out" << n << " ";    // 呼叫後：回來時才印
}

int main() {
    f(3);
    cout << "\n";
}
```

Python

```python
def f(n):
    if n == 0:
        return
    print("in" + str(n), end=" ")     # 呼叫前：往下走時印
    f(n - 1)
    print("out" + str(n), end=" ")    # 呼叫後：回來時才印

f(3)
print()
```

輸入	輸出
（不需要輸入）	in3 in2 in1 out1 out2 out3

追蹤遞迴的方法

- 每呼叫一次，就像在桌上疊一張新的紙（堆疊）：寫下這一層的參數和區域變數。
- **呼叫前**印的，照往下走的順序印；**呼叫後**印的，要等裡面全部做完才印，順序剛好**反過來**。
- 函式裡呼叫自己兩次以上，就畫成**遞迴樹**：樹上有幾個節點，就呼叫了幾次。

C++ 兩次遞迴呼叫：畫出遞迴樹

```cpp
#include <bits/stdc++.h>
using namespace std;

void g(int n) {
    if (n == 0) return;
    g(n - 1);
    cout << n << " ";
    g(n - 1);
}

int main() {
    g(3);
    cout << "\n";
}
```

Python

```python
def g(n):
    if n == 0:
        return
    g(n - 1)
    print(n, end=" ")
    g(n - 1)

g(3)
print()
```

輸入	輸出
（不需要輸入）	1 2 1 3 1 2 1

遞迴樹的一層	這層呼叫幾次	說明

g(3)	1	最上面，自己一次
g(2)	2	g(3) 呼叫兩次
g(1)	4	每個 g(2) 各呼叫兩次
g(0)	8	碰到終止條件，直接 return
合計	15	$1 + 2 + 4 + 8 = 2^4 - 1$

6. 費氏數列：呼叫次數爆炸

C++ 直接遞迴算費氏數列，數一數呼叫幾次

```cpp
#include <bits/stdc++.h>
using namespace std;

long long calls = 0;

long long fib(int n) {
    calls++;
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    for (int n : {10, 20, 30}) {
        calls = 0;
        long long v = fib(n);
        cout << n << ": fib=" << v << " calls=" << calls << "\n";
    }
}
```

Python

```python
calls = 0

def fib(n):
    global calls
    calls += 1
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

for n in (10, 20, 30):
    calls = 0
    v = fib(n)
    print(f"{n}: fib={v} calls={calls}")
```

輸入	輸出

（不需要輸入）	`10: fib=55 calls=177`
	`20: fib=6765 calls=21891`
	`30: fib=832040 calls=2692537`

- fib(n - 1) 和 fib(n - 2) 裡面又各自重算一大堆一樣的東西：fib(30) 光是 fib(2) 就算了 50 多萬次。
- 呼叫次數 = 2 × fib(n+1) - 1，大約每多 1 就變成 1.6 倍；n = 40 就要 3 億多次，n = 50 會跑好幾分鐘。
- 解法：**算過的答案存起來，下次直接拿**，這叫「記憶化」，是動態規劃的起點（後面單元會詳細教）。

C++ 記憶化：算過就記下來（預告）

```cpp
#include <bits/stdc++.h>
using namespace std;

long long memo[91];              // 0 代表還沒算過

long long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];      // 算過就直接拿
    return memo[n] = fib(n - 1) + fib(n - 2);
}

int main() {
    cout << fib(80) << "\n";
}
```

Python

```python
memo = {}

def fib(n):
    if n <= 1:
        return n
    if n in memo:                # 算過就直接拿
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]

print(fib(80))
```

輸入	輸出
（不需要輸入）	23416728348467685

每個 fib(k) 只真正算一次，fib(80) 一瞬間就算完。

7. 河內塔、分治與遞迴深度

C++ 河內塔：把 n 個盤子從 A 搬到 C

```cpp
#include <bits/stdc++.h>
using namespace std;

int moves = 0;

void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);   // 上面 n-1 個先搬到中間
    cout << n << ":" << from << "->" << to << "\n";
    moves++;                       // 最大的那個直接搬到目標
    hanoi(n - 1, via, from, to);   // 中間的 n-1 個再搬到目標
}

int main() {
    hanoi(3, 'A', 'B', 'C');
    cout << "total " << moves << "\n";
}
```

Python

```python
moves = 0

def hanoi(n, frm, via, to):
    global moves
    if n == 0:
        return
    hanoi(n - 1, frm, to, via)     # 上面 n-1 個先搬到中間
    print(f"{n}:{frm}->{to}")
    moves += 1                     # 最大的那個直接搬到目標
    hanoi(n - 1, via, frm, to)     # 中間的 n-1 個再搬到目標

hanoi(3, "A", "B", "C")
print("total", moves)
```

輸入	輸出
（不需要輸入）	1:A->C 2:A->B 1:C->B 3:A->C 1:B->A 2:B->C 1:A->C total 7

- 搬 n 個 = 搬 n-1 個 + 1 次 + 搬 n-1 個，所以次數 $T(n) = 2T(n-1) + 1 = 2^n - 1$。
- 10 個盤子要 1023 次，64 個盤子要 $2^{64} - 1$ 次：這就是「指數成長」。

C++ 分治：切兩半，各自解決再合併

```cpp
#include <bits/stdc++.h>
using namespace std;

int a[8] = {5, 2, 9, 1, 7, 3, 8, 6};

int best(int l, int r) {          // a[l..r] 的最大值
    if (l == r) return a[l];      // 只剩一個，它就是答案
    int m = (l + r) / 2;
    return max(best(l, m), best(m + 1, r));
}

int main() {
    cout << best(0, 7) << "\n";
}
```

Python

```python
a = [5, 2, 9, 1, 7, 3, 8, 6]

def best(l, r):                   # a[l..r] 的最大值
    if l == r:
        return a[l]               # 只剩一個，它就是答案
    m = (l + r) // 2
    return max(best(l, m), best(m + 1, r))

print(best(0, 7))
```

輸入	輸出
（不需要輸入）	9

分治三步：切成兩半 → 兩半各自遞迴解決 → 合併兩邊的答案。長度 n 的區間總共會呼叫 2n - 1 次。U08 的合併排序就是這個想法。

Python Python 的遞迴深度上限

```python
import sys
sys.setrecursionlimit(10000)      # 預設上限大約 1000 層

def depth(n):
    if n == 0:
        return 0
    return depth(n - 1) + 1

print(depth(5000))
```

輸入	輸出
（不需要輸入）	5000

沒有加第 2 行，depth(5000) 會出現 RecursionError。上限調太大、層數真的太深，程式還是會直接當掉。

遞迴會不會太深？

- C++ 深度到幾十萬層可能 stack overflow；Python 預設約 1000 層就 RecursionError，可用 sys.setrecursionlimit 放寬。
- 像 gcd、快速冪這種每次「減半」的遞迴，深度只有幾十層，很安全；每次只減 1 的遞迴，n 很大時就要改用迴圈。

讀完這份，你應該要會

- □ 會寫有參數、有回傳值的函式，分得清 print 和 return
- □ C++ 的 & 參考、Python 的 list 原地修改，都能改到外面；整數傳值改不到
- □ 分得清區域與全域；Python 要修改全域變數先寫 global
- □ 寫遞迴前先想好三要素：終止條件、縮小問題、組合答案
- □ 會畫遞迴樹數呼叫次數，知道呼叫後才印的會反過來
- □ 知道費氏數列直接遞迴會爆炸，河內塔要 $2^n - 1$ 步

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室

IG @jf_chuang

LINE 諮詢 jenfoo2021